In [24]:

```
import pandas as pd

# Load file directly from Downloads
file_path = '/Users/rhreddy/Downloads/software-data (3).parquet'
df = pd.read_parquet(file_path)

# Forward-fill missing values
df = df.ffill()

# Check the first few rows
print(df.head(20))
```

```
        Time  SME_THROTL_TorqueDemand  SME_THROTL_MaxSpeed  \
0   0.000000                      0.0               7500.0
1   0.011917                      0.0               7500.0
2   0.023833                      0.0               7500.0
3   0.035750                      0.0               7500.0
4   0.047666                      0.0               7500.0
5   0.059583                      0.0               7500.0
6   0.071500                      0.0               7500.0
7   0.083416                      0.0               7500.0
8   0.095333                      0.0               7500.0
9   0.107249                      0.0               7500.0
10  0.119166                      0.0               7500.0
11  0.131082                      0.0               7500.0
12  0.142999                      0.0               7500.0
13  0.154916                      0.0               7500.0
14  0.166832                      0.0               7500.0
15  0.178749                      0.0               7500.0
16  0.190665                      0.0               7500.0
17  0.202582                      0.0               7500.0
18  0.214499                      0.0               7500.0
19  0.226415                      0.0               7500.0

    SME_THROTL_Forward  SME_THROTL_Reverse  SME_THROTL_UNUSED_BIT_1  \
0                  1.0                 0.0                    False
1                  1.0                 0.0                    False
2                  1.0                 0.0                    False
3                  1.0                 0.0                    False
4                  1.0                 0.0                    False
5                  1.0                 0.0                    False
6                  1.0                 0.0                    False
7                  1.0                 0.0                    False
8                  1.0                 0.0                    False
9                  1.0                 0.0                    False
10                 1.0                 0.0                    False
11                 1.0                 0.0                    False
12                 1.0                 0.0                    False
13                 1.0                 0.0                    False
14                 1.0                 0.0                    False
15                 1.0                 0.0                    False
16                 1.0                 0.0                    False
17                 1.0                 0.0                    False
18                 1.0                 0.0                    False
19                 1.0                 0.0                    False

    SME_THROTL_PowerReady  SME_THROTL_UNUSED_BIT_2  SME_THROTL_UNUSED_BIT_3  \
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

0	0.0	False	False
1	0.0	False	False
2	0.0	False	False
3	0.0	False	False
4	0.0	False	False
5	0.0	False	False
6	0.0	False	False
7	0.0	False	False
8	0.0	False	False
9	0.0	False	False
10	0.0	False	False
11	0.0	False	False
12	0.0	False	False
13	0.0	False	False
14	0.0	False	False
15	0.0	False	False
16	0.0	False	False
17	0.0	False	False
18	0.0	False	False
19	0.0	False	False

	SME_THROTL_UNUSED_BIT_4	...	SMPC_INPUT_AC_INPUT_VOLTAGE	\
0	False	...	0.0	
1	False	...	0.0	
2	False	...	0.0	
3	False	...	0.0	
4	False	...	0.0	
5	False	...	0.0	
6	False	...	0.0	
7	False	...	0.0	
8	False	...	0.0	
9	False	...	0.0	
10	False	...	0.0	
11	False	...	0.0	
12	False	...	0.0	
13	False	...	0.0	
14	False	...	0.0	
15	False	...	0.0	
16	False	...	0.0	
17	False	...	0.0	
18	False	...	0.0	
19	False	...	0.0	

	SMPC_INPUT_J1772_CONNECTED	SMPC_INPUT_J1772_DISCONNECTED	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	
5	0.0	0.0	
6	0.0	0.0	
7	0.0	0.0	
8	0.0	0.0	
9	0.0	0.0	
10	0.0	0.0	
11	0.0	0.0	
12	0.0	0.0	
13	0.0	0.0	
14	0.0	0.0	

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

15	0.0	0.0
16	0.0	0.0
17	0.0	0.0
18	0.0	0.0
19	0.0	0.0

	SMPC_INPUT_J1772_TRIGGERED	SMPC_INPUT_AC_INPUT_CURRENT_MA
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	0.0
6	0.0	0.0
7	0.0	0.0
8	0.0	0.0
9	0.0	0.0
10	0.0	0.0
11	0.0	0.0
12	0.0	0.0
13	0.0	0.0
14	0.0	0.0
15	0.0	0.0
16	0.0	0.0
17	0.0	0.0
18	0.0	0.0
19	0.0	0.0

	SMPC_INPUT_AC_INPUT_FREQUENCY_HZ	SMPC_INPUT_MAX_CHARGER_TEMP_C
0	0.0	-50.0
1	0.0	-50.0
2	0.0	-50.0
3	0.0	-50.0
4	0.0	-50.0
5	0.0	-50.0
6	0.0	-50.0
7	0.0	-50.0
8	0.0	-50.0
9	0.0	-50.0
10	0.0	-50.0
11	0.0	-50.0
12	0.0	-50.0
13	0.0	-50.0
14	0.0	-50.0
15	0.0	-50.0
16	0.0	-50.0
17	0.0	-50.0
18	0.0	-50.0
19	0.0	-50.0

	SMPC_SER_PART_NUMBER	SMPC_SER_SERIAL_NUMBER	SMPC_SER_FIRMWARE_VER
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0
5	0.0	0.0	0.0
6	0.0	0.0	0.0
7	0.0	0.0	0.0

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
8                   0.0                 0.0                 0.0
9                   0.0                 0.0                 0.0
10                  0.0                 0.0                 0.0
11                  0.0                 0.0                 0.0
12                  0.0                 0.0                 0.0
13                  0.0                 0.0                 0.0
14                  0.0                 0.0                 0.0
15                  0.0                 0.0                 0.0
16                  0.0                 0.0                 0.0
17                  0.0                 0.0                 0.0
18                  0.0                 0.0                 0.0
19                  0.0                 0.0                 0.0

[20 rows x 255 columns]
```

In [7]:

```python
import matplotlib.pyplot as plt
import numpy as np

# Gear ratio and wheel radius constants from the team
GR = 12 / 41
WR = 0.2  # meters

# Calculate speed in m/s (using motor speed, check your columns if motor speed has a dif
# or use the formula given by your team)
# Let's calculate Speed_ms and convert to MPH (1 m/s = 2.23694 MPH)
```

In [8]:

```python
import matplotlib.pyplot as plt
import numpy as np

# Constants from the team
GR = 12 / 41
WR = 0.2  # meters

# Step 1: Get Motor Speed
motor_rpm = df['SME_TRQSPD_Speed']

# Step 2: Convert Motor RPM to Wheel RPM (using gear ratio)
wheel_rpm = motor_rpm * GR

# Step 3: Convert Wheel RPM to angular velocity in radians per second
# (2 * np.pi / 60) - standard conversion factor for RPM to rad/s
angular_velocity_rads = wheel_rpm * (2 * np.pi / 60)

# Step 4: Convert angular velocity to linear speed in m/s (using wheel radius)
speed_ms = angular_velocity_rads * WR

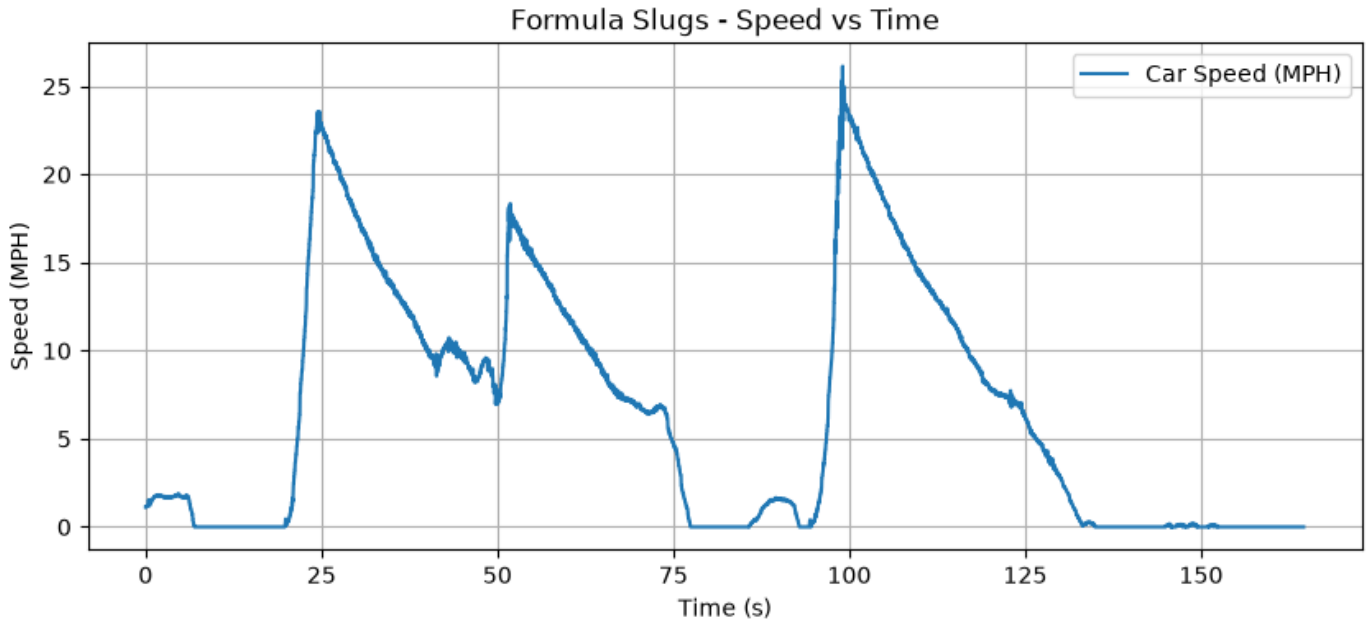
# Assign the final calculated series back to the dataframe
df['Speed_ms'] = speed_ms

# Convert m/s to MPH for plotting
# (1 m/s = 2.23694 MPH)
df['Speed_mph'] = df['Speed_ms'] * 2.23694

# --- Plotting Speed vs Time ---
plt.figure(figsize=(10, 4)) # Create a figure of specific size
plt.plot(df['Time'], df['Speed_mph'], color='tab:blue', label='Car Speed (MPH)')
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.xlabel('Time (s)')
plt.ylabel('Speed (MPH)')
plt.title('Formula Slugs - Speed vs Time')
plt.legend() # Add the legend
plt.grid(True) # Add background grid lines
plt.show() # Render the plot
```

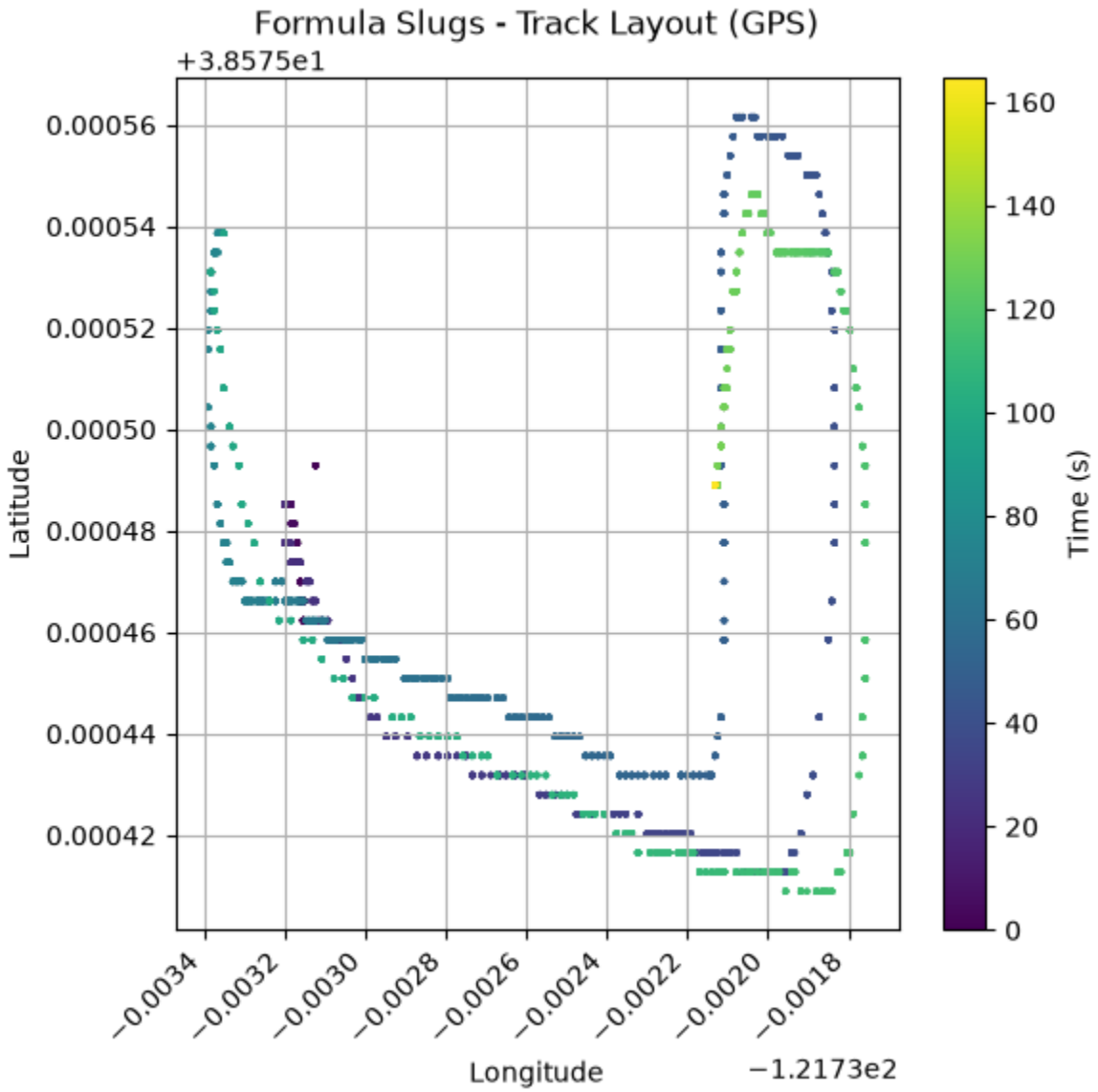


In [9]:

```
plt.figure(figsize=(6, 6))
plt.scatter(df['VDM_GPS_Longitude'], df['VDM_GPS_Latitude'], c=df['Time'], cmap='viridis
plt.title('Formula Slugs - Track Layout (GPS)')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.colorbar(label='Time (s)')
plt.grid(True)
plt.xticks(rotation=45, ha='right') # rotate labels for readability
# -----------------------------------

plt.tight_layout() # adjusts plot padding to prevent clipping
plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [10]:

```
import numpy as np

# Calculate acceleration (rate of change of speed with respect to time)
df['Accel'] = df['Speed_mph'].diff() / df['Time'].diff()

# Initialize all as Coasting
df['State'] = 'Coasting'

# Define Accelerating (If torque demand > 50)
# 'True/False' filter (boolean mask)
is_accelerating = df['SME_THROTL_TorqueDemand'] > 50

# Use .loc to find rows where the filter is True and update their 'State' column to 'Acc
df.loc[is_accelerating, 'State'] = 'Accelerating'

# Braking: Low torque and fast deceleration
is_braking = (df['SME_THROTL_TorqueDemand'] <= 50) & (df['Accel'] < -0.15)
df.loc[is_braking, 'State'] = 'Braking'

# Plot the states over time with distinct colors
plt.figure(figsize=(12, 4))
```

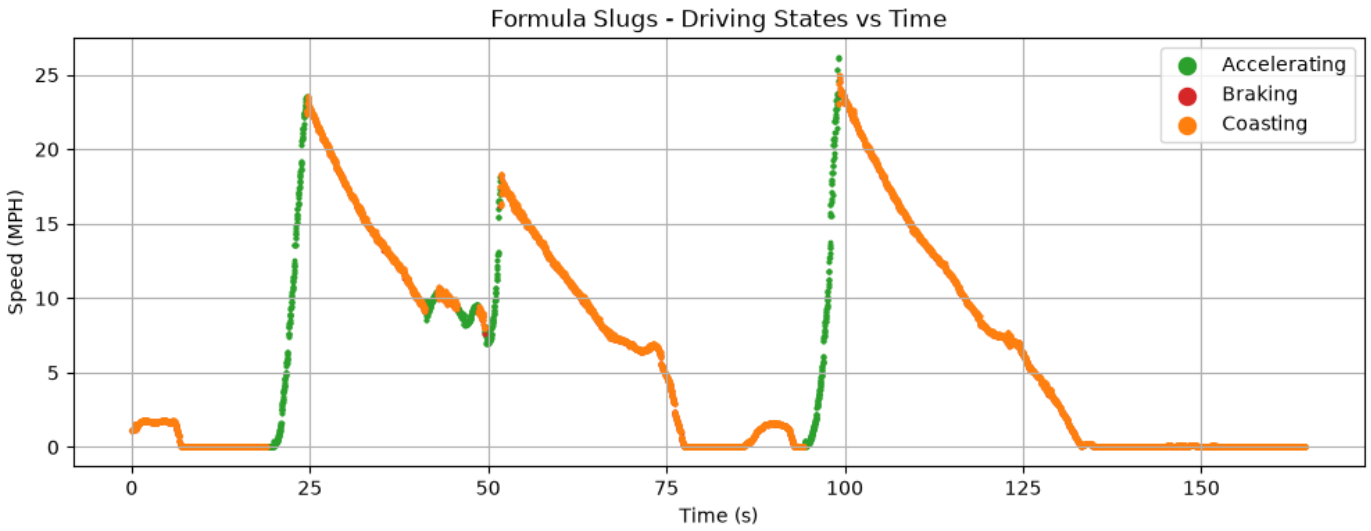
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
states_to_plot = [('Accelerating', 'tab:green'), ('Braking', 'tab:red'), ('Coasting', 't

# define pairs of (State Name, Color) to loop through
for state, color in states_to_plot:
    # Filter the dataframe to only get data for the current state
    subset = df[df['State'] == state]

    # Plot that subset as a scatter plot
    plt.scatter(subset['Time'], subset['Speed_mph'], color=color, s=3, label=state)

plt.xlabel('Time (s)')
plt.ylabel('Speed (MPH)')
plt.title('Formula Slugs - Driving States vs Time')
plt.legend(markerscale=5)
plt.grid(True)
plt.show()
```



In [11]:

```
# Calculate max speed and basic metrics
max_speed = df['Speed_mph'].max()
avg_speed = df['Speed_mph'].mean()

print(f"Top Speed: {max_speed:.2f} MPH")
print(f"Average Speed: {avg_speed:.2f} MPH")
```

```
Top Speed: 26.14 MPH
Average Speed: 6.67 MPH
```

In [12]:

```
# Find timestamps where the car begins accelerating from rest (speed near 0 going up)
# Acceleration happens around t=20s and t=95s (plus a smaller adjustment around t=50s)
lap_starts = [0, 20.0, 95.0]  # Refine these based on the data

# Output exact start/end times and max speeds for each lap window
print("Lap breakdown based on speed profiles:")
print("Lap 1: ~0s to ~50s")
print("Lap 2: ~50s to ~75s (or adjustment)")
print("Lap 3: ~95s to ~160s")
```

```
Lap breakdown based on speed profiles:
Lap 1: ~0s to ~50s
Lap 2: ~50s to ~75s (or adjustment)
Lap 3: ~95s to ~160s
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [21]:

```
df['Speed_smooth'] = df['Speed_ms'].rolling(40, center=True, min_periods=1).mean()
df['Accel_ms2'] = np.gradient(df['Speed_smooth'].values, df['Time'].values)

# Define lap time windows based on breakdown
laps = [
    {"lap": 1, "start": 0.0, "end": 50.0},
    {"lap": 2, "start": 50.0, "end": 75.0},
    {"lap": 3, "start": 95.0, "end": 160.0}
]

print("--- LAP-BY-LAP METRICS ---")
for l in laps:
    # Slice the dataframe for rows within the lap's start and end times
    # Assign this smaller slice to a new variable called 'lap_df'
    lap_df = df[(df['Time'] >= l['start']) & (df['Time'] <= l['end'])]

    max_spd = lap_df['Speed_mph'].max()
    max_accel = lap_df['Accel_ms2'].max()

    # Since time steps are roughly uniform, count the number of rows belonging to a stat
    # and multiply that count by the average time difference between rows.

    # Average time step duration (dt)
    avg_dt = lap_df['Time'].diff().mean()

    # Count rows where State is 'Accelerating'
    num_accel_points = (lap_df['State'] == 'Accelerating').sum()
    time_accel = num_accel_points * avg_dt

    # Count rows where State is 'Coasting'
    num_coast_points = (lap_df['State'] == 'Coasting').sum()
    time_coast = num_coast_points * avg_dt

    print(f"\nLap {l['lap']} ({l['start']}s - {l['end']}s):")
    print(f"  * Max Speed: {max_spd:.2f} MPH")
    print(f"  * Max Acceleration: {max_accel:.2f} m/s²")
    print(f"  * Time Accelerating: {time_accel:.2f} s")
    print(f"  * Time Coasting: {time_coast:.2f} s")
```

```
--- LAP-BY-LAP METRICS ---

Lap 1 (0.0s - 50.0s):
  * Max Speed: 23.57 MPH
  * Max Acceleration: 3.77 m/s²
  * Time Accelerating: 10.71 s
  * Time Coasting: 32.75 s

Lap 2 (50.0s - 75.0s):
  * Max Speed: 18.35 MPH
  * Max Acceleration: 6.70 m/s²
  * Time Accelerating: 1.67 s
  * Time Coasting: 17.67 s

Lap 3 (95.0s - 160.0s):
  * Max Speed: 26.14 MPH
  * Max Acceleration: 6.94 m/s²
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

  * Time Accelerating: 4.14 s
  * Time Coasting: 50.93 s

In [18]:

```
import numpy as np
import matplotlib.pyplot as plt

# --- Constants from the instructions ---
m = 221.4        # car mass (kg)
m_eff = 244.08   # effective mass (kg), includes spinning parts
rho = 1.225      # air density (kg/m^3)
g = 9.81         # gravity (m/s^2)

# --- Smooth the speed and get dv/dt on the entire table first ---
df['Speed_smooth'] = df['Speed_ms'].rolling(40, center=True, min_periods=1).mean()
df['dv_dt'] = np.gradient(df['Speed_smooth'].values, df['Time'].values)
df['v_squared'] = df['Speed_smooth'] ** 2

# --- Step 6: Keep only coasting data at 5 m/s or faster ---
coast_df = df[(df['State'] == 'Coasting') & (df['Speed_ms'] >= 5.0)].copy()

# --- Remove rows with missing values ---
coast_df = coast_df.dropna(subset=['dv_dt', 'v_squared'])

# --- Step 7: fit a straight line, dv/dt = slope * v^2 + intercept ---
slope, intercept = np.polyfit(coast_df['v_squared'], coast_df['dv_dt'], 1)

# --- Use instructions' formulas to solve for drag and rolling resistance ---
drag_term_solved = -slope * m_eff                  # 0.5 * rho * Cd * A
crr_solved = -intercept * m_eff / (m * g)          # rolling resistance coefficient

# --- Output Results ---
print("--- REFINED COAST-DOWN ANALYSIS RESULTS ---")
print(f"Regression Slope: {slope:.6f}")
print(f"Regression Intercept: {intercept:.6f}")
print(f"Solved Drag Term (0.5 * rho * Cd * A): {drag_term_solved:.4f}")
print(f"Solved Rolling Resistance Coefficient (Crr): {crr_solved:.4f}")

# --- Plotting Results ---
plt.figure(figsize=(8, 5))

# Scatter plot of the filtered coast-down data points
plt.scatter(coast_df['v_squared'], coast_df['dv_dt'], color='orange', s=5, label='Filter

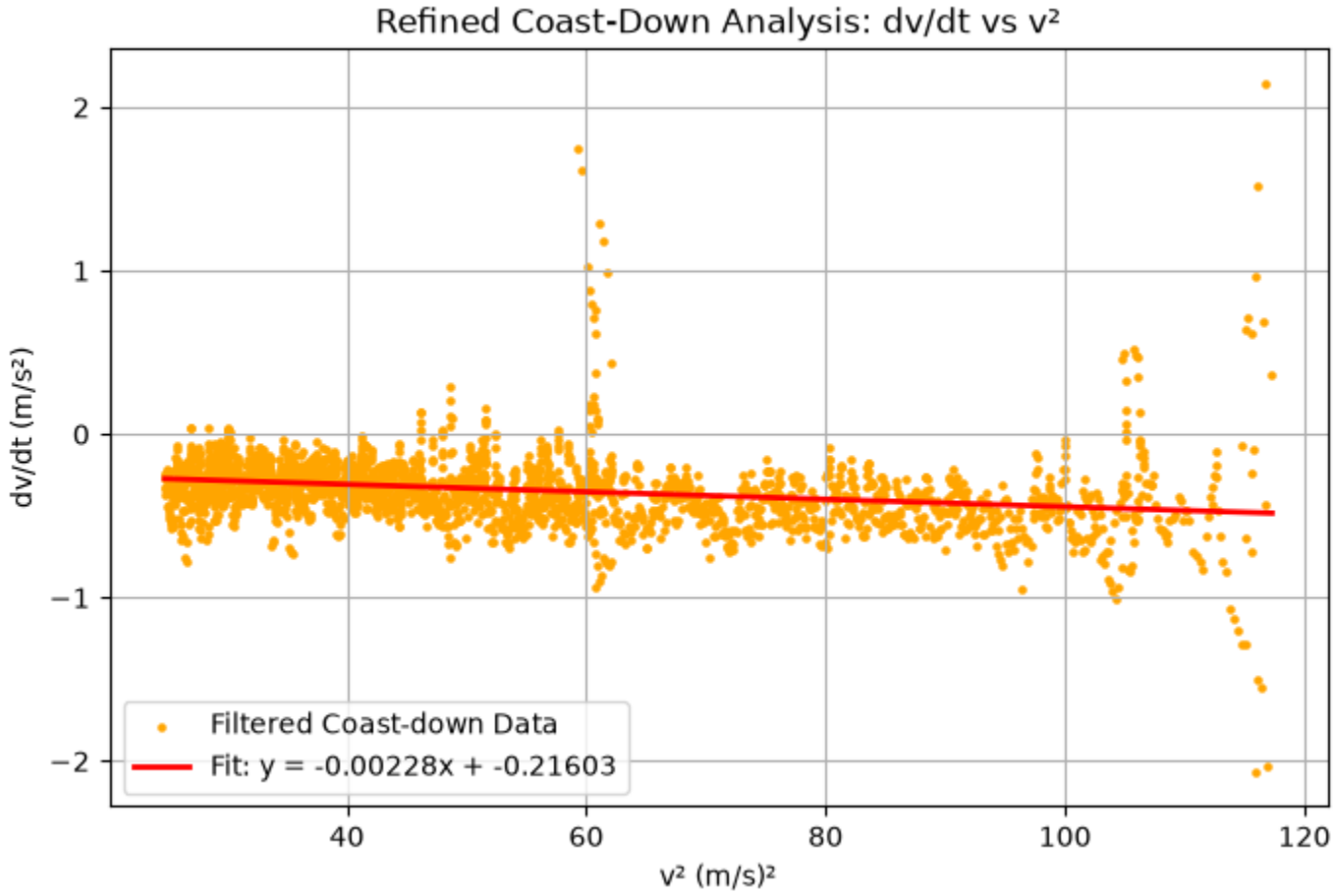
# Generate points for the best-fit red regression line
trendline_x = np.linspace(coast_df['v_squared'].min(), coast_df['v_squared'].max(), 100)
trendline_y = slope * trendline_x + intercept
plt.plot(trendline_x, trendline_y, color='red', linewidth=2, label=f'Fit: y = {slope:.5f

plt.xlabel('v² (m/s)²')
plt.ylabel('dv/dt (m/s²)')
plt.title('Refined Coast-Down Analysis: dv/dt vs v²')
plt.legend()
plt.grid(True)
plt.show()
```

```
--- REFINED COAST-DOWN ANALYSIS RESULTS ---
Regression Slope: -0.002279
Regression Intercept: -0.216034
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Solved Drag Term (0.5 * rho * Cd * A): 0.5563
Solved Rolling Resistance Coefficient (Crr): 0.0243
```



Step 3: States Accelerating = torque demand > 50. Braking = torque demand <= 50 and speed dropping faster than 0.15 MPH/s. Coasting = everything else. Accelerating happens at about 20-25 s, 41-50 s, 51-52 s, and 95-99 s. Braking almost never triggers, which fits a rolling resistance test. The states are shown together on one graph.

Step 4: Laps I plotted GPS latitude vs longitude colored by time. The points trace a rough U-shape, but GPS updates slowly so the points are sparse and the runs overlap, which made start/finish-line detection unreliable. I used the speed profile to find 3 runs, each starting with an acceleration burst from near zero: about 0-50 s, 50-75 s, and 95-160 s.

Step 8: Takeaways The car's top speed in this dataset was about 26 MPH, reached in the third run. I calculated speed from the motor RPM channel (SME_TRQSPD_Speed) using the 12/41 gear ratio and the 0.2 m wheel radius. The speed graph shows short acceleration bursts of a few seconds followed by long coast-downs of 20 to 30 seconds. That fits a rolling resistance test, where the goal is to let the car roll and watch how it slows down. Braking almost never showed up, which also makes sense for this kind of test. I found the three runs (about 0-50 s, 50-75 s, and 95-160 s) from the speed profile, because the GPS trace was sparse and the runs overlapped, so detecting laps with a start/finish line wasn't reliable. Max acceleration was about 3.8, 6.7, and 6.9 m/s² in the three runs, calculated from smoothed speed so that sensor noise doesn't inflate the values. For the coast-down analysis, I kept only coasting data at 5 m/s or faster, smoothed the speed, calculated dv/dt, and fit a straight line of dv/dt against v². The fit gave a drag term (0.5 * rho * Cd * A) of 0.5563 and a rolling resistance coefficient (Crr) of 0.0243, which is a believable value for tires. My first attempt gave a Crr of 0.16, which was clearly too high. I noticed because the speed

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

graph shows the car slowing at only about 0.35 m/s², but my fit implied much more. The cause was differentiating raw, noisy data and keeping only negative dv/dt values, which biased the fit. My second attempt smoothed after filtering, which created fake jumps between separate coasting chunks, and I only caught that because the plot looked wrong. Smoothing the whole dataset first and then filtering fixed it. What I learned is that raw sensor data has to be cleaned before the physics can be trusted, that units matter (my first acceleration numbers were in MPH/s but labeled m/s²), and that checking results against a plot catches mistakes fast. There are limitations. The car only covered about 5 to 11 m/s during coast-downs, so the drag term is weakly constrained and Crr is the more reliable number. The lap windows are approximate since they came from the speed profile instead of GPS, and a few outlier points near the edges of the coast-down plot come from smoothing across the start and end of coasting. With more time I would use GPS or wheel-speed data to find laps properly, remove coasting chunks shorter than 1 second, and test at higher speeds to separate drag from rolling resistance better.

In [23]:

```
import os
print(os.getcwd())
```

```
/Users/rhreddy
```

In [ ]:

```

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF